# Part 1: Early-signal decision (note)

**Deliverable:** `part1/out/predictions.csv` (`game_id, d7, d30, ltv30, alt, üst, karar`, plus D7/D30 intervals). **Reproduce:** `python3 part1/run_part1.py` (≈10 s). All numbers below come from that run. Throughout, **[measured]** = computed on held-out data; **[assumed]** = a judgement or extrapolation we cannot check with this data.

## 1. Data and definitions

- 60 games × 3 cohorts. G01–G45 have days 0–30 (training). G46–G60 have days 0–3 (forecast).
- Metrics are pooled over the 3 cohorts: retention(t) = Σ active(t) / Σ installs, and LTV30 = Σ revenue(days 0–30) / Σ installs. CPI is constant within a game.
- **Noise [measured]:** each cohort-day is about ±4% off a smooth curve on every day (cohorts of the same game disagree; 21% of days *rise* vs the day before). Pooled, about 2.2% remains, so even a perfect model has ~1.8% mean error. 83 day-0 rows show >100% retention. That is the visible half of symmetric noise, so we **kept the raw data**: dropping those rows doubled the LTV30 error (3.1% → 6.7%, because it removes only the high side), and capping changed nothing (`part1/analysis/cleaning_test.py`; charts: `part1/analysis/figs/outliers_day0.png`, `noise.png`, `cleaning_options.png`).

## 2. Method

- **Baseline:** per game, fit `retention = c·day^b` on days 1–3 and extrapolate. LTV30 = observed revenue days 0–3 + (revenue per active user, days 0–3) × Σ predicted retention days 4–30.
- **Why that LTV formula [measured]:** revenue per active user is flat over a game's life (days 4–30 average 1.01× the days 0–3 value, correlation 0.99), and a power law fitted on days 1–30 misses by only 2.3% (≈ the noise). So the curve *shape* is right. The hard part is estimating its two parameters from 3 noisy points.
- **Model:** instead of trusting the 3-point fit, learn the *true* day 1–30 parameters (c, b) from early signals: the 3-point fit itself, D1, D3, mean playtime days 0–3, and revenue per active user. Two ridge regressions (regularisation picked by internal CV) with 6 features and 45 games. The feature set was fixed before validation and not tuned on it. Genre was left out: mean decay slopes per genre only range from –0.59 to –0.52, with 6–15 games per genre, too few to learn a genre effect safely.

## 3. Validation

- **Leave-one-game-out (45 folds):** each game is predicted by a model trained on the other 44, using only its own days 0–3, which is exactly the real situation. We split by game, not by row, so a game's cohorts never sit on both sides of the split. K-fold with 45 games would waste data, and a time split is not possible (all games share the same calendar of days).

| held-out mean abs error | baseline | model | gain, 95% bootstrap CI | model better in |
|---|---|---|---|---|
| D7 | 4.5% | 3.5% | 1.1 pp [0.2, 1.9] | 29/45 games |
| D30 | 8.0% | 6.6% | 1.3 pp [–0.0, 2.7] | 28/45 |
| LTV30 | 3.8% | 3.1% | 0.8 pp [0.2, 1.4] | 29/45 |

- **Honest reading:** the model beats the baseline on D7 and LTV30. **On D30 the gain is not statistically clear** (the CI touches 0). The model's main win is removing the baseline's upward bias (D30 bias +2.5% → 0.0%). Both are within 2–5 pp of the noise floor, so there is little left to gain.
- **Intervals:** 90% intervals are the empirical 5th/95th percentiles of held-out errors (split-conformal style). Checked with a nested leave-one-out: **coverage 89% for all three targets [measured]**. Typical width: D7 ±9%, D30 ±15%, LTV30 ±8%.
- **What the intervals do NOT cover [assumed]:** they assume G46–G60 come from the same process as G01–G45 (their days 0–3 features fall inside the training range, which we checked). They cover forecast error only, not CPI changing at scale.

## 4. Decision rule

- **Threshold logic.** LTV30 is not lifetime value. On the training games, extending each game's own day 1–30 power law gives LTV180 ≈ 2.12× LTV30 and LTV365 ≈ 2.90× LTV30 (medians) **[assumed: extrapolation beyond day 30 with no data to check it; real curves usually steepen, so this is optimistic]**.
- **Continue** if the *lower* 90% bound of LTV30/CPI ≥ 1/2.12 = **0.47**: pays back the user cost within ~6 months even on the pessimistic forecast.
- **Stop** if the *upper* bound < 1/2.90 = **0.35**: cannot pay back within a year even on the optimistic forecast.
- **Retest** otherwise.
- **Cost of errors (why the gap between 0.35 and 0.47).** Stopping a good game is the more expensive mistake: a killed prototype rarely comes back, and the upside of a hit is large and unbounded. Continuing a bad game costs a few weeks of development, and the next test catches it. A retest costs one more ad test, about **$2.3k** (median installs × CPI in this data). So we stop only when a game fails even the lenient 1-year bar, and we continue only when it passes the stricter 6-month bar on its pessimistic forecast.
- **Backtest on G01–G45 [measured, held-out forecasts]:** of 14 truly good games (true ratio ≥ 0.47), 11 were continued, 3 sent to retest, and **none stopped**. Of 21 truly bad games (< 0.35), 17 were stopped, 4 sent to retest, and **none continued**. All 10 games in between went to retest. The price of zero wrong continue/stop calls is 17 retests out of 45.

**Result:** continue **G49, G51, G53, G55**. Retest **G52, G56, G57, G59**. Stop the other 7.

## 5. What I do not trust / limitations

1. **CPI drives the decision as much as LTV does** (in the test set, log-variance 0.24 vs 0.21). G49 ranks first mainly because its CPI is $0.11. Test CPIs come from small campaigns. If CPI is 15% higher at scale, G51 moves from continue to retest. The intervals do not include this.

2. **The 6/12-month multipliers are extrapolations**, so the thresholds inherit that assumption. They are better read as "a principled place to draw the line" than as a measured payback.
3. **Synthetic data:** curves are very clean power laws, so the intervals are narrow (±8% on LTV30). Real games have events, seasonality, and genre-specific monetisation, and intervals would need to be much wider.
4. **45 training games** is little. The interval tails rest on 2–3 games each.

## 6. AI tools used and how I checked them

Claude Code (Claude Opus 5.5) wrote the analysis scripts and the plots. I checked its output by: rerunning every script from a clean shell; recomputing the G01 example by hand (5250 installs, D7 = 550/5250); asking for every claim to be backed by a script and re-checking numbers when they changed (two small corrections were made this way: 20% → 21%, 2.3% → 2.2%); and testing its assumptions instead of accepting them (the drop/cap/keep experiment came from questioning a data-cleaning choice).